

# Лабораторная работа №1

## Генерация DAG и алгоритм Шимбелла

Теория графов

16 февраля 2026 г.

### Содержание

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Введение</b>                                          | <b>3</b> |
| <b>2</b> | <b>Задание 1: Генерация связного ациклического графа</b> | <b>3</b> |
| 2.1      | Теория . . . . .                                         | 3        |
| 2.2      | Алгоритм генерации DAG . . . . .                         | 3        |
| 2.3      | Матрица смежности . . . . .                              | 4        |
| 2.4      | Пример . . . . .                                         | 4        |
| <b>3</b> | <b>Задание 2: Метод Шимбелла</b>                         | <b>5</b> |
| 3.1      | Постановка задачи . . . . .                              | 5        |
| 3.2      | Алгоритм Шимбелла . . . . .                              | 5        |
| 3.3      | Пошаговый пример . . . . .                               | 5        |
| 3.3.1    | К = 1 (один переход) . . . . .                           | 5        |
| 3.3.2    | К = 2 (два перехода) . . . . .                           | 5        |
| 3.3.3    | К = 3 (три перехода) . . . . .                           | 6        |
| 3.4      | Результат . . . . .                                      | 6        |
| <b>4</b> | <b>Задание 3: Подсчёт маршрутов (Backtracking)</b>       | <b>7</b> |
| 4.1      | Постановка задачи . . . . .                              | 7        |
| 4.2      | Алгоритм Backtracking . . . . .                          | 7        |
| 4.3      | Пример . . . . .                                         | 8        |
| <b>5</b> | <b>Структура кода</b>                                    | <b>9</b> |
| 5.1      | Шаблонный класс Graph<T> . . . . .                       | 9        |
| 5.2      | Нормальное распределение . . . . .                       | 9        |
| 5.3      | Генератор DAG . . . . .                                  | 9        |
| 5.4      | Алгоритм Шимбелла . . . . .                              | 10       |

|          |                                |           |
|----------|--------------------------------|-----------|
| <b>6</b> | <b>Пример работы программы</b> | <b>10</b> |
| 6.1      | Генерация графа . . . . .      | 10        |
| 6.2      | Метод Шимбелла . . . . .       | 11        |
| 6.3      | Подсчёт маршрутов . . . . .    | 11        |
| <b>7</b> | <b>Заключение</b>              | <b>11</b> |

# 1. Введение

Цель лабораторной работы — изучить основные алгоритмы работы с графами:

- Генерация связного ациклического графа (DAG)
- Алгоритм Шимбелла для поиска путей фиксированной длины
- Подсчёт маршрутов методом backtracking

## 2. Задание 1: Генерация связного ациклического графа

### 2.1. Теория

**DAG (Directed Acyclic Graph)** — ориентированный ациклический граф. Граф  $G = (V, E)$ , где:

- $V = \{v_0, v_1, \dots, v_{n-1}\}$  — множество вершин
- $E \subseteq V \times V$  — множество рёбер
- Каждое ребро имеет вес  $w(i, j) \in R^+$

**Нормальное распределение** для генерации весов рёбер:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

где  $\mu$  — среднее значение,  $\sigma$  — стандартное отклонение.  
В нашей задаче:  $\mu = 10.0$ ,  $\sigma = 3.0$ .

### 2.2. Алгоритм генерации DAG

1. Создать  $n$  вершин:  $V = \{0, 1, 2, \dots, n-1\}$
2. Для каждой пары вершин  $(i, j)$ , где  $i < j$ :
  - С вероятностью  $p$  создать ребро  $i \rightarrow j$
  - Вес ребра:  $w(i, j) = |N(\mu, \sigma^2)|$  (модуль нормально распределённого значения)
3. Убедиться что граф связный (есть путь между любыми двумя вершинами)

## 2.3. Матрица смежности

Граф представляется матрицей смежности  $A_{n \times n}$ :

$$A[i][j] = \begin{cases} w(i, j), & i \rightarrow j \\ 0, & \text{иначе} \end{cases}$$

## 2.4. Пример

Для графа с 4 вершинами:

$$A = \begin{pmatrix} 0 & 8.5 & 12.3 & 0 \\ 0 & 0 & 9.7 & 11.2 \\ 0 & 0 & 0 & 7.8 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Это означает:

- $0 \rightarrow 1$  с весом 8.5
- $0 \rightarrow 2$  с весом 12.3
- $1 \rightarrow 2$  с весом 9.7
- $1 \rightarrow 3$  с весом 11.2
- $2 \rightarrow 3$  с весом 7.8

## 3. Задание 2: Метод Шимбелла

### 3.1. Постановка задачи

Найти кратчайшие и длиннейшие пути длины **ровно К рёбер** между всеми парами вершин.

**Важно:** К — это количество рёбер в пути, не количество вершин!

### 3.2. Алгоритм Шимбелла

Пусть  $D^{(k)}[i][j]$  — минимальная стоимость пути из  $i$  в  $j$  через ровно  $k$  рёбер.

**Рекуррентное соотношение:**

$$D^{(0)}[i][j] = A[i][j]$$

$$D^{(k)}[i][j] = \min_{p=0}^{n-1} (D^{(k-1)}[i][p] + A[p][j])$$

Для максимальных путей:

$$D_{\max}^{(k)}[i][j] = \max_{p=0}^{n-1} (D_{\max}^{(k-1)}[i][p] + A[p][j])$$

### 3.3. Пошаговый пример

Для графа:

$$A = \begin{pmatrix} 0 & 2 & 5 & 0 & 0 & 3 & 4 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

#### 3.3.1. К = 1 (один переход)

$D^{(1)} = A$  — просто начальная матрица смежности.

#### 3.3.2. К = 2 (два перехода)

Вычислим  $D^{(2)}[0][2]$  (путь  $0 \rightarrow ? \rightarrow 2$  длины 2):

$$D^{(2)}[0][2] = \min \{ D^{(1)}[0][0] + A[0][2] = \infty, D^{(1)}[0][1] + A[1][2] = 2 + 3 = 5, D^{(1)}[0][2] + A[2][2] = \infty \}$$

Путь:  $0 \rightarrow 1 \rightarrow 2$  с весом 5.

Вычислим  $D^{(2)}[0][3]$  (путь  $0 \rightarrow ? \rightarrow 3$  длины 2):

$$D^{(2)}[0][3] = \min \{ D^{(1)}[0][0] + A[0][3] = \infty, D^{(1)}[0][1] + A[1][3] = 2 + 4 = 6, D^{(1)}[0][2] + A[2][3] = \infty \}$$

Пути:  $0 \rightarrow 1 \rightarrow 3$  или  $0 \rightarrow 2 \rightarrow 3$ , оба с весом 6.

### 3.3.3. $K = 3$ (три перехода)

Вычислим  $D^{(3)}[0][3]$  (путь  $0 \rightarrow ? \rightarrow ? \rightarrow 3$  длины 3):

$$D^{(3)}[0][3] = \min \{ D^{(2)}[0][0] + A[0][3] = \infty, D^{(2)}[0][1] + A[1][3] = \infty, D^{(2)}[0][2] + A[2][3] = 5 + 1 \}$$

Путь:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$  с весом 6.

## 3.4. Результат

Программа выводит две матрицы:

- Матрица минимальных путей длины  $K$
- Матрица максимальных путей длины  $K$

Элемент  $D^{(k)}[i][j]$  показывает минимальную/максимальную стоимость пути из  $i$  в  $j$  через ровно  $k$  рёбер.

## 4. Задание 3: Подсчёт маршрутов (Backtracking)

### 4.1. Постановка задачи

Дано: начальная вершина  $S$  и конечная вершина  $T$ .

Найти:

- Существует ли маршрут из  $S$  в  $T$ ?
- Сколько различных маршрутов из  $S$  в  $T$ ?
- Вывести все маршруты

### 4.2. Алгоритм Backtracking

**Маршрут** — последовательность вершин  $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ , где:

- $v_0 = S$  (начало)
- $v_k = T$  (конец)
- Для всех  $i$ : существует ребро  $v_i \rightarrow v_{i+1}$
- Все вершины различны (нет циклов)

**Псевдокод:**

```
1 def findRoutes(current, target, path, visited, routes):
2     # Добавляем текущую вершину в путь
3     path.append(current)
4     visited[current] = True
5
6     # Если достигли цели - сохраняем маршрут
7     if current == target:
8         routes.append(path.copy())
9     else:
10        # Пробуем все соседние вершины
11        for neighbor in adjacentVertices(current):
12            if not visited[neighbor]:
13                findRoutes(neighbor, target, path,
14                           visited, routes)
15
16    # Backtrack: убираем вершину из пути
17    path.pop()
18    visited[current] = False
```

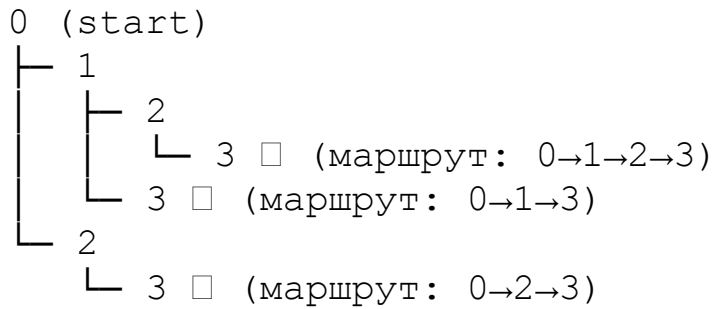
### 4.3. Пример

Для графа:

$$A = (0) 110001100010000$$

Найти все маршруты из  $S = 0$  в  $T = 3$ :

**Дерево рекурсии:**



**Результат:**

- Существует маршрут: **ДА**
- Количество маршрутов: **3**
- Маршруты:
  1.  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$
  2.  $0 \rightarrow 1 \rightarrow 3$
  3.  $0 \rightarrow 2 \rightarrow 3$



## 5. Структура кода

### 5.1. Шаблонный класс Graph<T>

```
1 template <typename T>
2 class Graph {
3 private:
4     int numVertices;
5     std::vector<std::vector<T>> adjMatrix;
6
7 public:
8     Graph(int n);
9     void addEdge(int from, int to, T weight);
10    T getWeight(int from, int to) const;
11    void printAdjMatrix() const;
12    // ...
13};
```

Шаблон позволяет использовать:

- Graph<double> для весовых графов
- Graph<int> для графов с целыми весами (например, пропускные способности в сетях)

### 5.2. Нормальное распределение

```
1 class NormalDistribution : public Distribution {
2 private:
3     double mean;      // среднее значение (mu)
4     double stddev;    // стандартное отклонение (sigma)
5
6 public:
7     NormalDistribution(double mu, double sigma);
8     double generate() override;
9};
```

### 5.3. Генератор DAG

```
1 template <typename T>
2 class DAGGenerator {
3 private:
4     Distribution* distribution;
5
6 public:
7     DAGGenerator(Distribution* dist);
8     Graph<T> generateDAG(int numVertices);
9};
```

## 5.4. Алгоритм Шимбелла

```
1 template <typename T>
2 class Shimbela {
3 private:
4     Graph<T>& graph;
5
6 public:
7     Shimbela(Graph<T>& g);
8
9     // Найдите минимальные пути длины k
10    std::vector<std::vector<T>> findMinPaths(int k);
11
12    // Найдите максимальные пути длины k
13    std::vector<std::vector<T>> findMaxPaths(int k);
14 };
```

## 6. Пример работы программы

### 6.1. Генерация графа

введите количество вершин N: 4

параметры нормального распределения:

среднее ( $\mu$ ) = 10.0

стандартное отклонение ( $\sigma$ ) = 3.0

граф успешно сгенерирован!

матрица смежности:

|   | 0 | 1    | 2     | 3     |
|---|---|------|-------|-------|
| 0 | - | 8.50 | 12.30 | -     |
| 1 | - | -    | 9.70  | 11.20 |
| 2 | - | -    | -     | 7.80  |
| 3 | - | -    | -     | -     |

список рёбер:

0 -> 1 (вес: 8.50)  
0 -> 2 (вес: 12.30)  
1 -> 2 (вес: 9.70)  
1 -> 3 (вес: 11.20)  
2 -> 3 (вес: 7.80)

## 6.2. Метод Шимбелла

введите количество рёбер К (длина пути) : 2

вычисление минимальных путей длины 2 рёбер...

минимальные пути длины 2:

|   | 0 | 1 | 2     | 3     |
|---|---|---|-------|-------|
| 0 | - | - | 18.20 | 19.70 |
| 1 | - | - | -     | 17.50 |
| 2 | - | - | -     | -     |
| 3 | - | - | -     | -     |

вычисление максимальных путей длины 2 рёбер...

максимальные пути длины 2:

|   | 0 | 1 | 2     | 3     |
|---|---|---|-------|-------|
| 0 | - | - | 18.20 | 20.10 |
| 1 | - | - | -     | 17.50 |
| 2 | - | - | -     | -     |
| 3 | - | - | -     | -     |

## 6.3. Подсчёт маршрутов

введите начальную вершину S: 0

введите конечную вершину T: 3

подсчёт маршрутов из вершины 0 в вершину 3...

существует маршрут: ДА

количество различных маршрутов: 3

все найденные маршруты:

1. 0 -> 1 -> 2 -> 3
2. 0 -> 1 -> 3
3. 0 -> 2 -> 3

## 7. Заключение

В ходе лабораторной работы были изучены:

- Генерация  $\square$ случайных взвешенных DAG с нормальным распределением
- Алгоритм Шимбелла для поиска путей фиксированной длины

- Метод backtracking для перебора маршрутов
- Работа с шаблонными классами в C++

Все алгоритмы реализованы в модульной структуре для переиспользования в последующих лабораторных работах (потокосые сети, эйлеровы и гамильтоновы графы и т.д.).